

OWASP ASVS & OWASP Top 10 Mapping Report

Vulnerable Flask Application Findings - Enterprise DevSecOps Project

Prepared By	Jagriti Banerjee
Project	Enterprise DevSecOps Security Lab
Assessment Focus	Flask application vulnerabilities, SAST/SCA evidence, OWASP ASVS alignment, and OWASP Top 10 risk mapping
Standards Basis	OWASP ASVS 5.0.0 and OWASP Top 10:2025
Environment	Private VM lab using Docker, Kubernetes Kind, GitHub Actions, Bandit, pip-audit, and Trivy

6	3	4	2
Mapped Findings	Injection-class Issues	ASVS Chapters Covered	CI/CD Evidence Sources

This document maps the intentionally vulnerable Flask application findings to OWASP Application Security Verification Standard controls and OWASP Top 10 risk categories. It is written as a professional AppSec crosswalk that can be used by recruiters, reviewers, or technical evaluators to understand how each vulnerability relates to secure development requirements and industry-recognized risk categories.

Confidentiality Notice: This report is based only on a private lab environment and does not target third-party or production systems.

Document Guide

Section	Purpose
1. Standards Basis	Explains the ASVS and Top 10 versions used for mapping.
2. Application Scope	Defines tested Flask routes, evidence sources, and assumptions.
3. Executive Mapping Summary	Summarizes findings by severity, OWASP Top 10 category, and ASVS chapter.
4. Finding-to-Standard Crosswalk	Maps each finding to OWASP ASVS controls, Top 10 categories, CWE, evidence, and remediation priority.
5. Detailed Finding Analysis	Provides technical explanation, impact, remediation, and retest criteria for each finding.
6. Verification and Retest Plan	Gives practical validation steps after remediation.
7. Evidence Appendix	Includes selected screenshots from the private VM lab.
8. References	Lists standards and project evidence sources used.

The report intentionally separates awareness mapping from technical control validation. OWASP Top 10 is used to explain broad business risk categories, while OWASP ASVS is used to map findings to concrete security verification requirements.

1. Standards Basis

OWASP ASVS 5.0.0 was selected as the primary control mapping framework because ASVS is designed to provide a basis for testing web application technical security controls and secure development requirements. The OWASP Top 10:2025 was used as the awareness-level risk taxonomy for executive and recruiter-friendly categorization.

Standard	Use in This Report	Why It Matters
OWASP ASVS 5.0.0	Maps each finding to specific verification requirements such as injection prevention, template injection protection, dependency governance, secure configuration, and logging.	ASVS gives concrete testable controls that go beyond high-level risk labels.
OWASP Top 10:2025	Groups findings into broad web application risk categories such as Injection, Software Supply Chain Failures, Security Misconfiguration, and Logging/Exceptional Conditions.	Top 10 provides concise risk communication for reviewers and non-specialist stakeholders.
CWE	Provides weakness identifiers such as CWE-89 for SQL Injection and CWE-78 for OS Command Injection.	CWE helps connect findings to standardized weakness classes and SAST/SCA terminology.

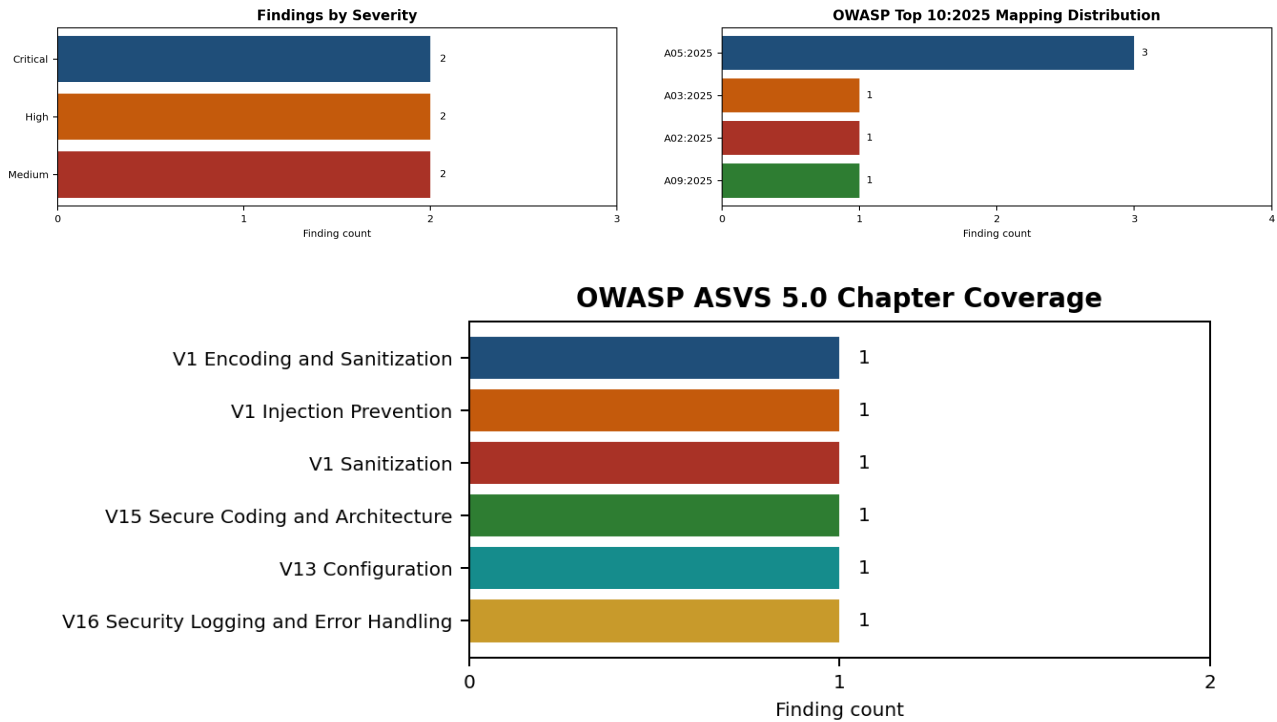
Versioning note: ASVS requirement identifiers are written with the explicit version prefix, for example v5.0.0-1.2.5. This avoids ambiguity if ASVS identifiers change in future releases.

2. Application Scope

Component	Observed Implementation / Evidence
Framework	Python Flask application located at src/app/app.py.
Database	SQLite users.db initialized by src/app/init_db.py.
Routes Reviewed	/user, /ping, /hello, /health and home route.
SAST Evidence	Bandit report detected B608 SQL injection vector, B602 shell=True, B404 subprocess import, and B104 bind-all-interfaces.
SCA Evidence	pip-audit report identified vulnerable direct and transitive Python dependencies.
Lab Assumption	The app is intentionally vulnerable for learning and private lab validation. Production status is not claimed.

3. Executive Mapping Summary

The vulnerable Flask app findings are dominated by injection risk, followed by supply chain, configuration, and logging/error-handling control gaps. This is consistent with the lab design: the app intentionally includes insecure query construction, shell execution, unsafe template rendering, and outdated dependencies so the DevSecOps workflow can detect and document them.



Metric	Result	Interpretation
Total mapped findings	6	Findings are limited to vulnerable Flask app and supporting SAST/SCA evidence.
Highest risk category	A05:2025 - Injection	Three findings map directly to injection-class weaknesses.
Most important ASVS chapter	V1 Encoding and Sanitization	Core control family for SQL injection, command injection, and SSTI.
Primary remediation theme	Remove unsafe interpreter use	Use parameterized SQL, safe subprocess calls, static templates, validation, and dependency updates.

4. Finding-to-Standard Crosswalk

The following matrix maps each finding to OWASP Top 10:2025, ASVS 5.0.0 requirement identifiers, severity, CWE class, and remediation priority. ASVS is used as the control basis, while OWASP Top 10 provides the high-level risk grouping.

ID	Finding	Severity	OWASP Top 10:2025	OWASP ASVS 5.0.0	CWE	Priority
APP-01	SQL Injection in /user Endpoint	Critical	A05:2025 - Injection	v5.0.0-1.2.4, v5.0.0-1.1.1	CWE-89	P1
APP-02	OS Command Injection in /ping Endpoint	Critical	A05:2025 - Injection	v5.0.0-1.2.5, v5.0.0-1.3.3, v5.0.0-13.2.4	CWE-78	P1
APP-03	Server-Side Template Injection Risk in /hello Endpoint	High	A05:2025 - Injection	v5.0.0-1.3.7, v5.0.0-1.3.5, v5.0.0-3.2.2	CWE-1336	P2
APP-04	Vulnerable Python Dependencies Identified by pip-audit	High	A03:2025 - Software Supply Chain Failures	v5.0.0-15.1.1, v5.0.0-15.1.2, v5.0.0-15.2.1, v5.0.0-15.2.4	CWE-1104 / CWE-937 / CWE-1035	P2
APP-05	Debug and Production Configuration Exposure Risk	Medium	A02:2025 - Security Misconfiguration	v5.0.0-13.4.2, v5.0.0-13.4.6, v5.0.0-15.2.3	CWE-489 / CWE-215	P3
APP-06	Insufficient Security Logging for AppSec Attack Attempts	Medium	A09:2025 - Security Logging and Alerting Failures / A10:2025 - Mishandling of Exceptional Conditions	v5.0.0-16.2.1, v5.0.0-16.3.3, v5.0.0-16.3.4, v5.0.0-16.4.1	CWE-778 / CWE-117	P3

Important interpretation: A finding may map to multiple ASVS controls because one weakness can violate more than one secure development requirement. For example, command injection maps to OS command injection prevention, contextual sanitization, and outbound communication allowlisting.

5. Detailed Finding Analysis

Each finding includes the vulnerable pattern, OWASP mapping, technical impact, recommended remediation, and retest criteria. The report treats the vulnerable state as an intentional lab condition and focuses on professional documentation of the weakness and its controls.

APP-01: SQL Injection in /user Endpoint		Critical / CVSS 9.1
Route / Component	/user?id=1	
OWASP Top 10:2025	A05:2025 - Injection	
OWASP ASVS 5.0.0	v5.0.0-1.2.4, v5.0.0-1.1.1	
ASVS Area	V1 Encoding and Sanitization - Injection Prevention	
CWE	CWE-89	
Status	Open in vulnerable lab version; remediation documented	

Summary: The endpoint builds a SQL query with f-string interpolation of user-controlled input. The application is intentionally vulnerable for lab testing, but the finding represents a production-critical design that can expose, modify, or destroy data.

Vulnerable Pattern:

```
query = f"SELECT id, username, role FROM users WHERE id = '{user_id}'"
```

Representative Test / Attack Pattern: id=1' OR '1'='1

Evidence: Manual payload returned multiple database rows; Bandit flagged B608 at app.py line 38.

Security Impact	• Unauthorized data retrieval from the users table • Bypass of intended row-level query logic • Potential data modification or destruction if write queries are added • Useful primitive for chained attacks against application data
Remediation Actions	• Replace string-built SQL with parameterized queries or ORM query builders. • Validate id as an integer before use. • Use least-privileged database accounts even in development. • Add negative unit tests for common SQL injection payloads.
Retest Criteria	• Submit normal id=1 and confirm only one expected row is returned. • Submit payload id=1' OR '1'='1 and confirm zero expansion or a validation error. • Rerun Bandit and verify B608 is no longer present for the route.

APP-02: OS Command Injection in /ping Endpoint		Critical / CVSS 9.3
Route / Component	/ping?host=127.0.0.1	
OWASP Top 10:2025	A05:2025 - Injection	
OWASP ASVS 5.0.0	v5.0.0-1.2.5, v5.0.0-1.3.3, v5.0.0-13.2.4	
ASVS Area	V1 Injection Prevention / V13 Backend Communication Configuration	
CWE	CWE-78	
Status	Open in vulnerable lab version; remediation documented	

Summary: The endpoint passes untrusted user input into a shell command with shell=True. This permits command separators and arbitrary shell syntax to be executed inside the container context.

Vulnerable Pattern:

```
subprocess.run(f"ping -c 1 {host}", shell=True, capture_output=True, text=True)
```

Representative Test / Attack Pattern: host=127.0.0.1;whoami

Evidence: Manual payload executed whoami through shell metacharacter injection; Bandit flagged B602 and B404.

Security Impact	• Arbitrary command execution inside the application container • Local file enumeration and environment discovery • Potential credential or secret exposure if mounted into the container • Potential pivot point if container isolation is weak
Remediation Actions	• Remove shell=True and pass command arguments as a list. • Validate host using ipaddress or a strict hostname allowlist. • Prefer an application-level reachability check instead of shelling out. • Run the container as non-root and keep read-only filesystem controls enabled.
Retest Criteria	• Submit allowed IP address and verify intended ping behavior. • Submit shell metacharacters such as ;, &&, and confirm they are rejected. • Rerun Bandit and verify B602 is closed.

APP-03: Server-Side Template Injection Risk in /hello Endpoint

High / CVSS 8.2

Route / Component	/hello?name=Jagriti
OWASP Top 10:2025	A05:2025 - Injection
OWASP ASVS 5.0.0	v5.0.0-1.3.7, v5.0.0-1.3.5, v5.0.0-3.2.2
ASVS Area	V1 Sanitization / V3 Web Frontend Security
CWE	CWE-1336
Status	Open in vulnerable lab version; remediation documented

Summary: The endpoint constructs a template string using the name parameter and renders it with `render_template_string`. In Flask/Jinja contexts, building templates from user input can enable template expression evaluation or content injection depending on payload and configuration.

Vulnerable Pattern:

```
template = f"<h1>Hello {name}</h1>"; return render_template_string(template)
```

Representative Test / Attack Pattern: `{{7*7}}` or similar template expression probes

Evidence: Code review identified `render_template_string` receiving a template built from untrusted input.

Security Impact	• Template expression evaluation risk • Exposure of internal objects depending on Jinja sandbox and context • HTML/script injection if output is not escaped safely • Potential route to remote code execution in unsafe template configurations
Remediation Actions	• Never build templates from user-controlled input. • Use static templates and pass user input as a variable. • Use automatic escaping and validate/sanitize display fields. • Add tests for Jinja expression payloads and HTML payloads.
Retest Criteria	• Submit template probes and confirm they render as inert text. • Submit HTML/script payloads and confirm safe escaping. • Review code to confirm <code>render_template_string</code> is not used with untrusted template content.

APP-04: Vulnerable Python Dependencies Identified by pip-audit

High / CVSS 8.0

Route / Component	requirements.txt
OWASP Top 10:2025	A03:2025 - Software Supply Chain Failures
OWASP ASVS 5.0.0	v5.0.0-15.1.1, v5.0.0-15.1.2, v5.0.0-15.2.1, v5.0.0-15.2.4
ASVS Area	V15 Secure Coding and Architecture - Dependencies
CWE	CWE-1104 / CWE-937 / CWE-1035
Status	Open in vulnerable lab version; remediation documented

Summary: The project intentionally pins outdated packages to demonstrate Software Composition Analysis. Vulnerable direct and transitive dependencies increase the risk of known exploit paths, denial of service, request smuggling, or data exposure depending on the vulnerable component.

Vulnerable Pattern:

Flask==2.3.3; requests==2.19.1; gunicorn==21.2.0

Representative Test / Attack Pattern: Exploit known CVEs in outdated direct or transitive dependencies

Evidence: pip-audit detected vulnerable dependencies, including Flask, requests, gunicorn, idna and urllib3; 25 vulnerability records were present in the report.

Security Impact	- Known public vulnerabilities may be exploitable without source-code weaknesses. - Transitive dependency risk may be missed without SCA and SBOM coverage. - Patch latency increases risk when remediation time frames are not defined. - Supply chain posture is weakened without trusted source and inventory controls.
Remediation Actions	- Upgrade vulnerable direct dependencies to fixed versions. - Regenerate lock files and SBOM after dependency updates. - Add pip-audit as a blocking or risk-gated CI/CD job. - Define risk-based remediation SLAs for critical, high, medium, and low dependency vulnerabilities.
Retest Criteria	- Run pip-audit -r src/app/requirements.txt and confirm no unresolved high or critical findings. - Review SBOM to confirm updated dependency versions. - Verify CI/CD job fails or gates when new high-risk dependency vulnerabilities are introduced.

APP-05: Debug and Production Configuration Exposure Risk

Medium / CVSS 6.4

Route / Component	Application runtime configuration
OWASP Top 10:2025	A02:2025 - Security Misconfiguration
OWASP ASVS 5.0.0	v5.0.0-13.4.2, v5.0.0-13.4.6, v5.0.0-15.2.3
ASVS Area	V13 Configuration / V15 Secure Coding and Architecture
CWE	CWE-489 / CWE-215
Status	Partially mitigated in container/Kubernetes deployment; app-level control still documented

Summary: The lab application can enable debug mode through an environment variable and binds to all interfaces for container operation. In production, debug mode must never be enabled and externally exposed development functionality should be removed or protected.

Vulnerable Pattern:

```
app.run(host="0.0.0.0", port=5000, debug=debug_mode)
```

Representative Test / Attack Pattern: Trigger debug console or error details if misconfigured in production

Evidence: Code review identified Flask app binding to 0.0.0.0 and environment-controlled debug mode; Bandit flagged B104 for binding to all interfaces.

Security Impact	- Information leakage through debug stack traces or version details - Unexpected exposure of development features - Broader network exposure if runtime is reachable from untrusted networks - Chaining potential with injection or dependency vulnerabilities
Remediation Actions	- Hard-disable debug mode for production builds. - Use Gunicorn or another production WSGI server instead of Flask debug server. - Restrict network exposure using Kubernetes Service, NetworkPolicy, ingress controls, and firewall rules. - Add CI/CD checks to detect debug flags and development-only code.
Retest Criteria	- Confirm FLASK_DEBUG is false or ignored in production container. - Confirm application runs under Gunicorn in the final Docker image. - Confirm unauthenticated external networks cannot directly reach the Flask debug server.

APP-06: Insufficient Security Logging for AppSec Attack Attempts

Medium / CVSS 5.8

Route / Component	/user, /ping, /hello
OWASP Top 10:2025	A09:2025 - Security Logging and Alerting Failures / A10:2025 - Mishandling of Exceptional Conditions
OWASP ASVS 5.0.0	v5.0.0-16.2.1, v5.0.0-16.3.3, v5.0.0-16.3.4, v5.0.0-16.4.1
ASVS Area	V16 Security Logging and Error Handling
CWE	CWE-778 / CWE-117
Status	Detection implemented at runtime layer with Falco; app-layer logging recommended

Summary: The DevSecOps lab includes runtime detection with Falco, but the Flask app itself does not generate structured security logs for injection attempts or validation bypass attempts. Application-layer security logs improve incident investigation and correlation.

Vulnerable Pattern:

No explicit security logger or validation-failure event path in `src/app/app.py`

Representative Test / Attack Pattern: Repeated SQLi/command injection attempts without high-signal app-layer events

Evidence: Application routes return responses but do not log validation failures, suspicious payloads, or security control bypass attempts at the application layer.

Security Impact	• Reduced forensic detail during investigation • Harder detection of slow probing or abuse patterns • Limited visibility into validation bypass attempts • Potential log injection if user input is logged later without encoding
Remediation Actions	• Add structured security logging for rejected inputs and suspicious payloads. • Avoid logging secrets or full malicious payloads; mask or hash risky values. • Use centralized log forwarding and correlation with Falco/Prometheus events. • Add tests that verify logging occurs for rejected injection patterns.
Retest Criteria	• Submit rejected SQLi and command injection payloads. • Confirm structured logs include timestamp, route, event type, and severity without sensitive values. • Confirm log output is safe against log injection and can be parsed by downstream tools.

6. Consolidated Remediation and Retest Plan

The remediation plan prioritizes exploitability and blast radius. Injection fixes should be implemented first because they directly affect confidentiality, integrity, and availability. Dependency fixes should be managed through CI/CD gating and SBOM maintenance. Configuration and logging improvements should be completed before any production-like exposure.

Phase	Controls	Actions	Retest Evidence
P1 - Injection closure	ASVS 1.2.4, 1.2.5, 1.3.7	Replace string-built SQL, remove shell=True, stop dynamic template construction from user input.	Negative payload tests fail safely; Bandit injection findings closed.
P2 - Dependency governance	ASVS 15.1.1, 15.1.2, 15.2.1	Update vulnerable packages, regenerate SBOM, enforce pip-audit in CI/CD.	pip-audit report shows no unresolved high/critical vulnerabilities.
P2 - Configuration hardening	ASVS 13.4.2, 13.4.6, 15.2.3	Disable debug mode, use production WSGI runtime, limit exposure through network controls.	Runtime proof shows Gunicorn, no debug console, and intended service exposure only.
P3 - Logging and monitoring	ASVS 16.2.1, 16.3.3, 16.3.4	Add structured security logs for rejected inputs and unexpected errors.	Security events appear in logs without secrets or raw unsafe payloads.

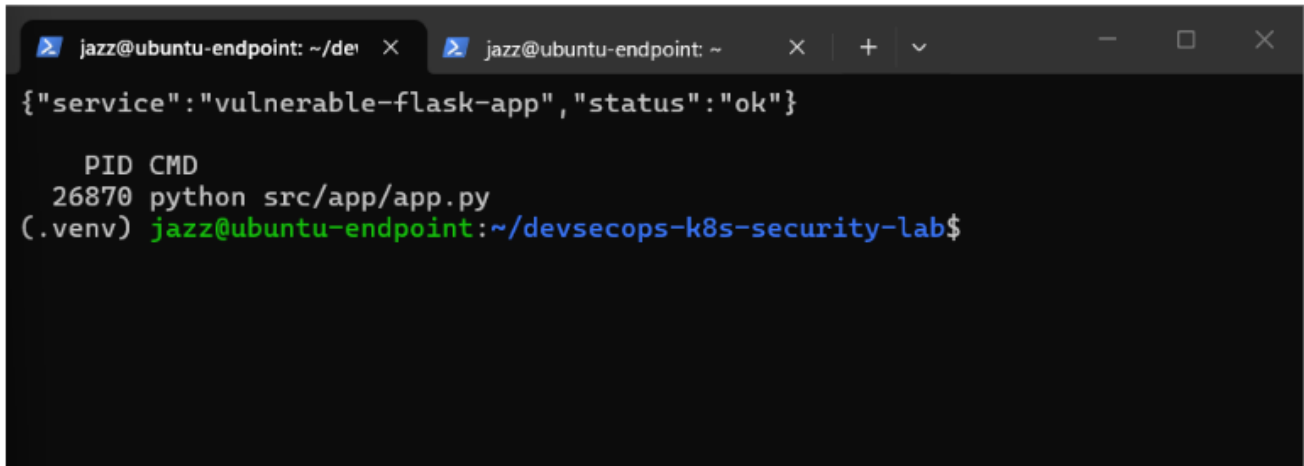
7. ASVS Control Interpretation Notes

ASVS Control	Applied Interpretation in This Lab
v5.0.0-1.2.4	Database queries must use parameterized queries, ORM query builders, or equivalent controls to prevent SQL and database injection.
v5.0.0-1.2.5	Operating system calls must not concatenate untrusted input into shell commands. Argument arrays and contextual allowlisting are expected.
v5.0.0-1.3.7	Templates must not be built from untrusted input; user-controlled values must be passed as variables into static templates.
v5.0.0-13.4.2	Debug modes must be disabled in production because they can leak internals or expose debugging features.
v5.0.0-15.1.2	A software inventory or SBOM should track third-party libraries and trusted sources.
v5.0.0-16.3.3	Attempts to bypass input validation or security controls should be logged for detection and investigation.

8. Evidence Appendix

The following screenshots were selected from the private VM lab evidence set. They support the application security findings and demonstrate that testing was performed in a controlled local environment.

App Running

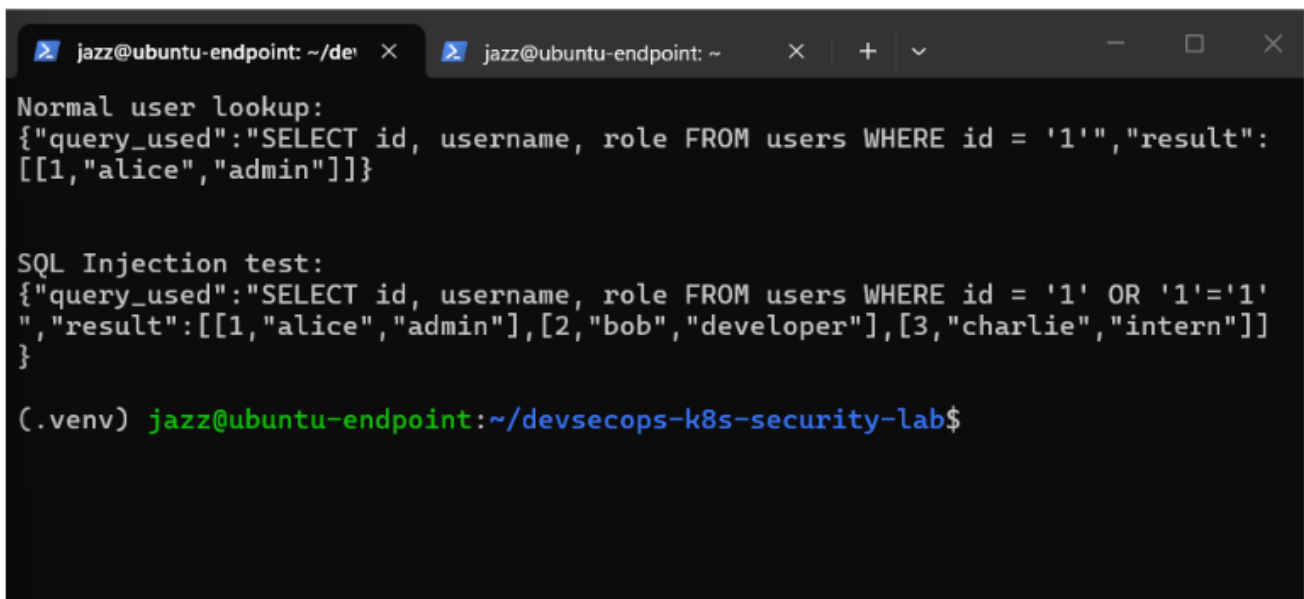


```
jazz@ubuntu-endpoint: ~/de  X  jazz@ubuntu-endpoint: ~  X  +  v  -  □  X
{"service": "vulnerable-flask-app", "status": "ok"}

PID CMD
26870 python src/app/app.py
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure: App Running captured from Enterprise DevSecOps private VM lab evidence.

SQL Injection Proof



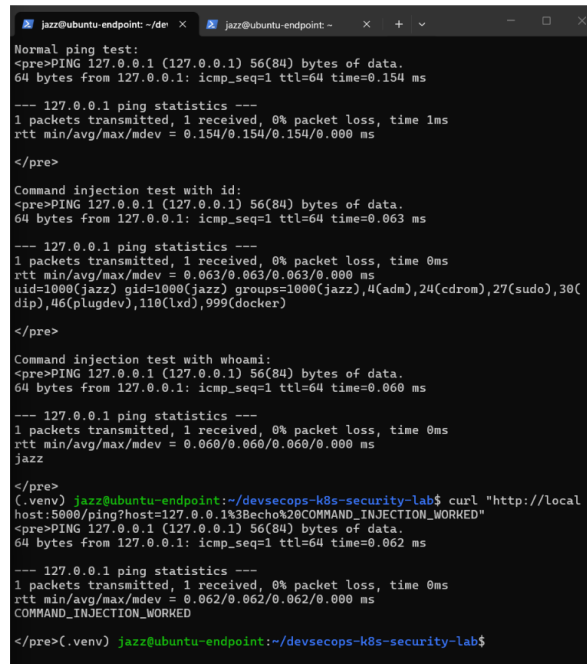
```
jazz@ubuntu-endpoint: ~/de  X  jazz@ubuntu-endpoint: ~  X  +  v  -  □  X
Normal user lookup:
{"query_used": "SELECT id, username, role FROM users WHERE id = '1'", "result": [[1, "alice", "admin"]]}

SQL Injection test:
{"query_used": "SELECT id, username, role FROM users WHERE id = '1' OR '1'='1'", "result": [[1, "alice", "admin"], [2, "bob", "developer"], [3, "charlie", "intern"]]}

(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure: SQL Injection Proof captured from Enterprise DevSecOps private VM lab evidence.

Command Injection Proof

A terminal window titled 'jazz@ubuntu-endpoint: ~' showing a series of ping tests and command injection attempts. The first test is a normal ping to 127.0.0.1. The second test is a command injection attempt using 'id' as the command, which returns user and group information. The third test is another command injection attempt using 'whoami', which returns 'jazz'. The fourth test is a command injection attempt using a curl command to trigger a local host ping, which also returns 'jazz'. The terminal output is as follows:

```
jazz@ubuntu-endpoint: ~  
Normal ping test:  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.154 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 1ms  
rtt min/avg/max/mdev = 0.154/0.154/0.154/0.000 ms  
</pre>  
  
Command injection test with id:  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.063 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.063/0.063/0.063/0.000 ms  
uid=1000(jazz) gid=1000(jazz) groups=1000(jazz),4(adm),24(cdrom),27(sudo),30(dip),46(plugindev),118(lxd),999(docker)  
</pre>  
  
Command injection test with whoami:  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.060 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.060/0.060/0.060/0.000 ms  
jazz  
</pre>  
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$ curl "http://localhost:5000/ping?host=127.0.0.1&3Becho%20COMMAND_INJECTION_WORKED"  
<pre>PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.062 ms  
  
--- 127.0.0.1 ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.062/0.062/0.062/0.000 ms  
COMMAND_INJECTION_WORKED  
</pre>(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$
```

Figure: Command Injection Proof captured from Enterprise DevSecOps private VM lab evidence.

Bandit SAST Findings

```

CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b692_subprocess_
popen_with_shell_equals_true.html
Location: src/app/app.py:58:13
54         f"ping -c 1 {host}",
55         shell=True,
56         capture_output=True,
57         text=True
58     )
59     return f"<pre>{result.stdout}\n{result.stderr}</pre>"
60
61
>> Issue: [B104:hardcoded_bind_all_interfaces] Possible binding to all interf
aces.
Severity: Medium Confidence: Medium
CWE: CWE-695 (https://cwe.mitre.org/data/definitions/695.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b104_hardcoded_b
ind_all_interfaces.html
Location: src/app/app.py:73:17
72     debug_mode = os.getenv("FLASK_DEBUG", "false").lower() == "true"
73     app.run(host="0.0.0.0", port=5000, debug=debug_mode)

Code scanned:
Total lines of code: 70
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
Undefined: 0
Low: 1
Medium: 2
High: 1
Total issues (by confidence):
Undefined: 0
Low: 1
Medium: 1
High: 2
Files skipped (0):
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Figure: Bandit SAST Findings captured from Enterprise DevSecOps private VM lab evidence.

pip-audit Dependency Findings

```

Found 25 known vulnerabilities in 5 packages
Name      Version ID          Fix Versions
-----
Flask     2.3.3   CVE-2026-27205  3.1.3
requests  2.19.1  PYSEC-2018-28  2.20.0
requests  2.19.1  PYSEC-2023-74  2.31.0
requests  2.19.1  PYSEC-2023-74  2.31.0
requests  2.19.1  CVE-2024-35195  2.32.0
requests  2.19.1  CVE-2024-47681  2.32.4
requests  2.19.1  CVE-2026-25645  2.33.0
gunicorn  21.2.0  CVE-2024-1135   22.0.0
gunicorn  21.2.0  CVE-2024-6827   22.0.0
idna      2.7     PYSEC-2024-60   3.7
idna      2.7     PYSEC-2024-60   3.7
idna      2.7     CVE-2026-45489  3.15
urllib3   1.23    PYSEC-2019-133  1.24.2
urllib3   1.23    PYSEC-2019-132  1.24.3
urllib3   1.23    PYSEC-2028-148  1.25.9
urllib3   1.23    PYSEC-2023-192  1.26.17, 2.0.6
urllib3   1.23    PYSEC-2023-192  1.26.17, 2.0.6
urllib3   1.23    PYSEC-2023-212  1.26.18, 2.0.7
urllib3   1.23    PYSEC-2023-207  1.24.2
urllib3   1.23    PYSEC-2023-212  1.26.18, 2.0.7
urllib3   1.23    PYSEC-2026-141  2.7.0
urllib3   1.23    CVE-2024-37891  1.26.19, 2.2.2
urllib3   1.23    CVE-2025-50181  2.5.0
urllib3   1.23    CVE-2025-66471  2.6.0
urllib3   1.23    CVE-2026-21401  2.6.3
(.venv) jazz@ubuntu-endpoint:~/devsecops-k8s-security-lab$

```

Figure: pip-audit Dependency Findings captured from Enterprise DevSecOps private VM lab evidence.

CI/CD SCA Job Evidence

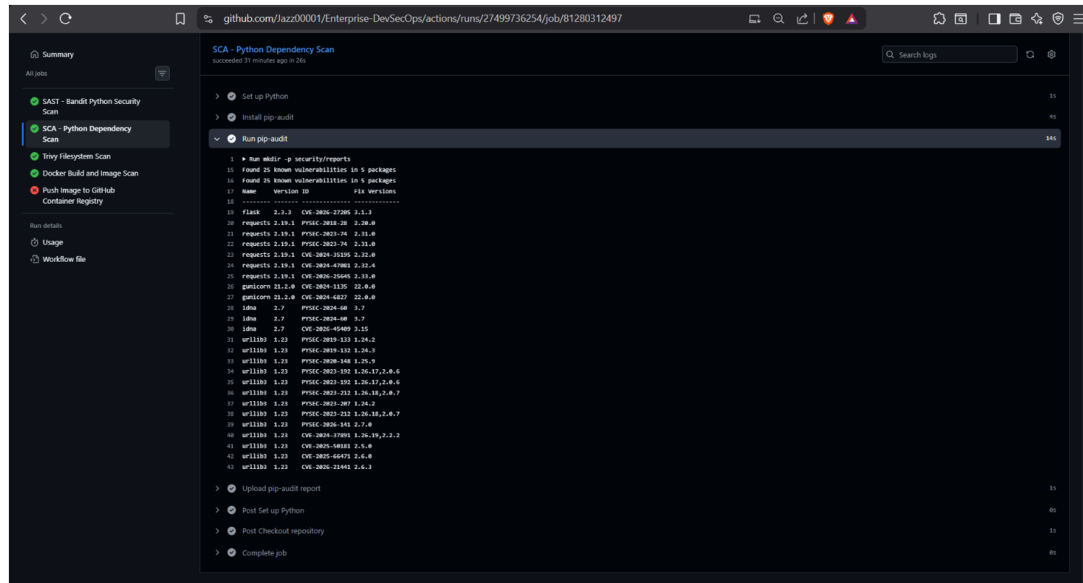


Figure: CI/CD SCA Job Evidence captured from Enterprise DevSecOps private VM lab evidence.

9. References

[1]	OWASP Application Security Verification Standard (ASVS) Project - purpose and standard overview.
[2]	OWASP ASVS 5.0.0 GitHub repository - latest stable release and version-specific requirements.
[3]	OWASP ASVS V1 Encoding and Sanitization - injection prevention and template injection controls.
[4]	OWASP ASVS V13 Configuration - debug mode and unintended information leakage controls.
[5]	OWASP ASVS V15 Secure Coding and Architecture - third-party library inventory, remediation time frames, and dependency controls.
[6]	OWASP ASVS V16 Security Logging and Error Handling - security logging, event logging, error handling, and log protection controls.
[7]	OWASP Top 10:2025 - current web application risk awareness categories.
[8]	OWASP Top 10:2025 A05 Injection, A03 Software Supply Chain Failures, A02 Security Misconfiguration, A09 Security Logging and Alerting Failures, and A10 Mishandling of Exceptional Conditions.
[9]	Project evidence: src/app/app.py, src/app/requirements.txt, Bandit report, pip-audit report, and screenshot evidence from Enterprise-DevSecOps-main.zip.

Document limitations: This report is an educational AppSec mapping report for a deliberately vulnerable lab application. It should not be interpreted as an external penetration test against any third-party environment.